

Where the CVFEM operators reach

Exactness, macro-element locality and the coarse space, drawn in two dimensions

September 16, 2026

Two different properties

Exact is a statement about *which terms* an operator contains. The Jacobian is

$$J_{\text{exact}} = A_{\text{frozen}} + KW^{-1}G.$$

The matrix-free Jacobian that FGMRES and Newton apply contains both terms, at every node. The assembled operators — the Galerkin coarse levels and the Vanka patches — contain A_{frozen} at every node and $KW^{-1}G$ at none.

Macro-element local is a statement about *which node pairs* a term couples. The element-wise assembly stores a sum of macro-element matrices, so it can hold an entry (i, m) only if some macro-element contains both i and m . Every entry of A_{frozen} satisfies that. Some entries of $KW^{-1}G$ do not, and those are the ones drawn in red below.

1 The mesh

The semi-structured mesh is an unstructured mesh of macro-elements, each refined uniformly. Figure 1 is the two-dimensional analogue used throughout: a triangle tiled by three quadrilateral macro-elements E_0, E_1, E_2 meeting at a vertex of valence three — a vertex no structured grid has — each refined into 4×4 micro-cells. Inside one macro-element the nodes form a lattice; across macro-elements they are shared through global node ids, and neighbouring lattices are oriented differently. Every set drawn in this note is computed from that connectivity, through global ids, by `python/cvfem_locality_figures.py`.

2 The three factors

2.1 The formulas, on the drawing

Every quantity below has a place in Figure 2, and the formulas use the figure's colours: green for faces and K , amber for gradients, G and W , blue for the control volume and A_{frozen} , and red for a node that no macro-element shares with i . Two meshes share the nodes: the *elements* Ω_c (the micro-cells, solid edges, tinted grey), which carry the shape functions N_a and every gradient, and the *control volumes* V_i (dotted blue edges, tinted blue), one per node, over which the equations are balanced. Node i owns V_i . Inside an element Ω_c , the boundary of V_i consists of two sub-control-volume faces —, each running from the element centre to the midpoint of an edge (i, j) . A face s carries an area vector $\mathbf{a}_s$ and the edge vector $\mathbf{d}_s = \mathbf{x}_j - \mathbf{x}_i$ (both drawn in panel a). $\sigma_{is} = \pm 1$ orients face s out of V_i .

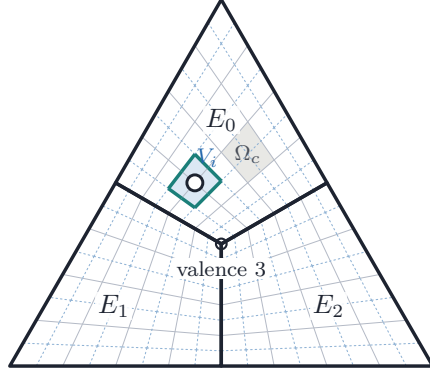


Figure 1: Two meshes on one set of nodes. The *elements* are the micro-cells (—, one tinted grey, Ω_c): the finite-element shape functions, gradients and cell areas live on them, and they tile each macro-element (thick). The *control volumes* are their dual (---, one tinted blue, V_i): one around each node, bounded by sub-control-volume faces that run from an element's centre to its edge midpoints. The balance equations are integrated over control volumes; every face s — of V_i lies inside one element and separates i $\circ$ from one edge neighbour. The circled vertex is shared by three macro-elements.

The residual. The mass flux through face s is the averaged velocity flux minus the Rhie–Chow correction,

$$\dot{m}_s = \rho \bar{\mathbf{u}}_s \cdot \mathbf{a}_s - c_s \left[(p_j - p_i) - \frac{1}{2} (\mathbf{g}_i + \mathbf{g}_j) \cdot \mathbf{d}_s \right], \quad (1)$$

and the continuity residual of node i sums it over the faces of V_i , $R_i^p = \sum_{s \in \partial V_i} \sigma_{is} \dot{m}_s$. Here $\mathbf{g}_n$ is the reconstructed nodal pressure gradient: the gradients of the micro-cells around n (shaded amber in panel b), weighted by their areas,

$$\mathbf{g}_n = \frac{1}{W_n} \sum_{c \ni n} |\Omega_c| \nabla p|_c, \quad W_n = \sum_{c \ni n} |\Omega_c|, \quad \nabla p|_c = \sum_{a \in c} p_a \nabla N_a|_c. \quad (2)$$

The Jacobian. Differentiating (1) in a pressure direction δp gives two groups of terms:

$$\delta R_i^p = \underbrace{\sum_{s \in \partial V_i} \sigma_{is} c_s (\delta p_i - \delta p_j)}_{A_{\text{frozen}}: \text{ nodes } i, j \text{ of one element}} + \underbrace{\sum_{s \in \partial V_i} \sigma_{is} \frac{1}{2} c_s \mathbf{d}_s \cdot (\delta \mathbf{g}_i + \delta \mathbf{g}_j)}_{K W^{-1} G: \text{ through } \delta \mathbf{g}}. \quad (3)$$

The first group, together with the velocity columns, is A_{frozen} : every term of face s reads only the corners of the element that contains s (panel a). The second group is where the gradient enters, and by (2) $\delta \mathbf{g}_n$ is linear in δp . Written as matrices,

$$[G]_{nm} = \sum_{c \ni n, c \ni m} |\Omega_c| \nabla N_m|_c, \quad [W]_{nn} = W_n, \quad [K]_{in} = \sum_{\substack{s \in \partial V_i \\ n \in \{i, j_s\}}} \frac{1}{2} \sigma_{is} c_s \mathbf{d}_s^T, \quad (4)$$

where j_s is the far end of face s . The momentum rows receive the same K multiplied by the upwind weights of face s . An entry of the product is a sum over paths through the gradient node:

$$[K W^{-1} G]_{im} = \sum_n \underbrace{[K]_{in}}_{\text{face } s \text{ of } V_i} \underbrace{W_n^{-1}}_{\text{area around } n} \underbrace{[G]_{nm}}_{\text{element } \Omega_{c'} \ni n, m}. \quad (5)$$

Panel (b) draws one term of this sum: the face s between i and n gives $[K]_{in}$ (green arrow), W_n is the total amber area around n — 4 elements from two macro-elements — and the darker element $\Omega_{c'}$ contains both n and m , giving $[G]_{nm}$ (dashed amber arrow). The node m lies in the neighbouring macro-element, which does not contain i .

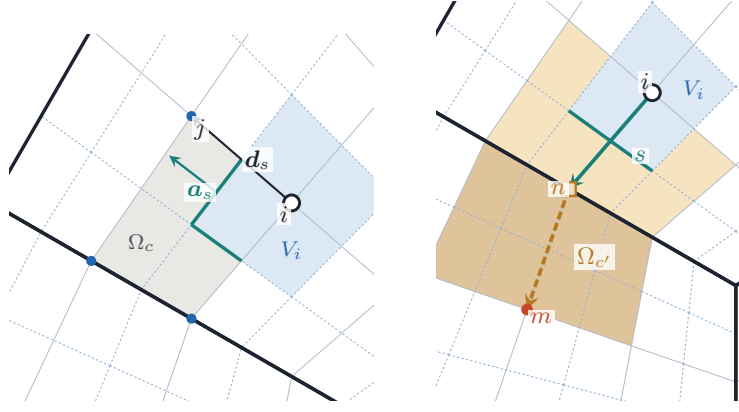


Figure 2: The formulas on the mesh, zoomed. (a) A frozen entry, Eq. (3) first group (A_{frozen}): in the element Ω_c (grey, solid edges) the two faces — of the control volume V_i (blue, dotted edges) separate i $\circ$ from its edge neighbours; each face term reads only the four corners $\bullet$ of Ω_c . The arrows are d_s from i to j and the face's area vector a_s . (b) One path of Eq. (5): face s of V_i reads the gradient at n $\square$ ($[K]_{in}$, $\rightarrow$); g_n averages the shaded cells around n on both sides of the macro-element boundary (thick line; W_n); the element $\Omega_{c'}$ contains n and m $\bullet$ ($[G]_{nm}$, $\rightarrow$). No macro-element contains both i and m , so a macro-element matrix cannot hold this entry.

2.2 The reach of each factor

The Jacobian therefore has two parts. A_{frozen} treats g as given. The remaining part differentiates through g and factors as $K W^{-1} G$, Eq. (4):

- G maps nodal pressures to area-weighted cell gradients summed at each node;
- W^{-1} divides by the nodal area — a diagonal, which reaches nothing;
- K maps the gradient at the two ends of each face into the rows the face flux enters: continuity, and momentum through the upwind weights.

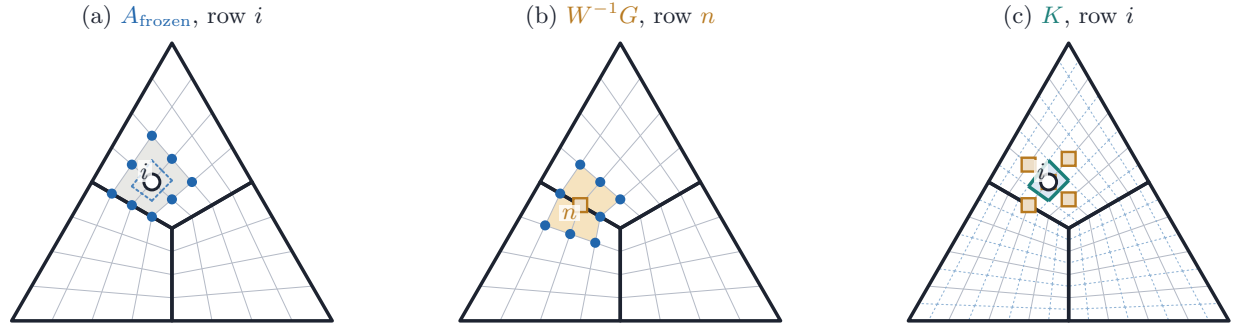


Figure 3: Row reach of each factor on the unstructured mesh. (a) A_{frozen} : every face term of the control volume (dotted outline) reads only the element it lies in, so row i $\circ$ reaches the 9 nodes $\bullet$ of the shaded elements. (b) $W^{-1}G$ at a node n $\square$ on the edge shared by E_0 and E_1 : it averages the 4 shaded elements from *both* macro-elements and reads their 9 nodes. (c) K , row i : the faces — of i 's control volume read the gradient at i and at its edge neighbours, 5 gradient nodes $\square$ — one of which is the shared node of panel (b).

Each sparse factor reaches one ring of nodes. A_{frozen} , G and K are each a sum of micro-cell contributions, so each is a sum of macro-element matrices. The difficulty is only in their product.

3 The product $KW^{-1}G$

Row i of the product reaches every node m that the gradient at some n in K 's row reads. The second ring comes from chaining the two one-ring factors through n . Figure 4 colours each reached node by whether a macro-element matrix could hold the entry.

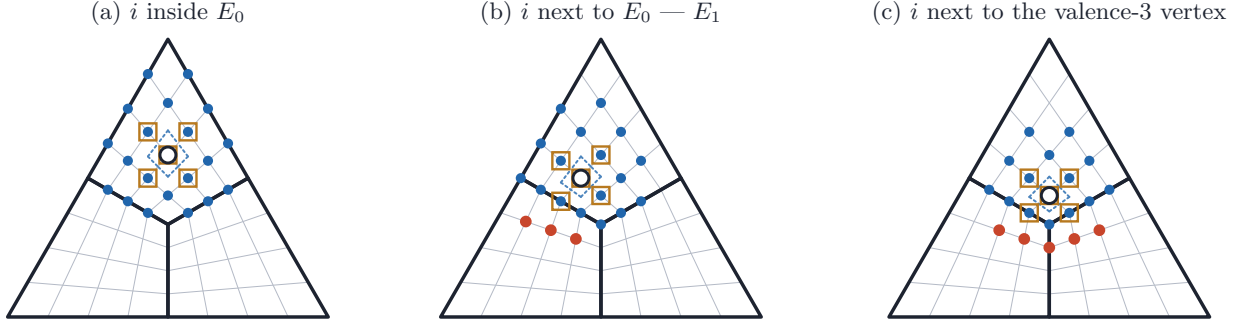


Figure 4: Row i of $KW^{-1}G$, through the gradient nodes of K 's row. $\bullet$ some macro-element contains both i and m ; $\bullet$ none does. (a) i inside E_0 : 21 nodes, 0 red. (b) One step from the edge shared with E_1 : 21 nodes, 3 red — the gradient at the shared node reads the cells of E_1 . (c) One step from the valence-3 vertex: 20 nodes, 5 red, reaching into both neighbours. A_{frozen} reaches 9, 9 and 9 nodes at the same three rows, none red.

This is exactly what the element-locality gate in the driver measures: a direction placed on the interior of one macro-element produces a response in its neighbours only through this term. The matrix-free Jacobian includes the red entries, because its gradient pass and its face pass both run over every macro-element and sum at shared nodes. The element-wise assembly omits the whole term — the blue entries as well as the red ones — and could only ever add the blue.

3.1 How the exact operator is applied

The product $KW^{-1}G$ is never formed. The matrix-free Jacobian applies its two factors in two passes, exactly as the path sum (5) suggests: first $W^{-1}G$ acts on the pressure of the direction, producing a nodal gradient g^q at every node (Algorithm 1); then the face loop applies K by reading that gradient at the two ends of each face (Algorithm 2). Both passes run macro-element by macro-element and sum shared nodes with the deterministic two-pass scatter, so the gradient at a node on a macro-element boundary contains every element around it — which is how the red entries enter the action. Algorithm 2 follows `sscvfem_apply`, `sscvfem_apply_macro_local_hoisted` and `sscvfem_action_hoisted`; the three-dimensional code has the same structure with twelve faces per element.

Algorithm 1 Nodal gradient $\mathbf{g} = \mathbf{W}^{-1}\mathbf{G}q$ (sscvfem.nodal_grad_strided), Eq. (2)

```

1: function NODALGRADIENT( $q$ )
2:    $\mathbf{g}_n \leftarrow \mathbf{0}$  for every node  $n$ 
3:   for all macro-elements  $E$  in parallel do
4:     gather  $q$  at the nodes of  $E$ 
5:     for all elements  $\Omega_c \subset E$  do
6:        $\gamma_c \leftarrow |\Omega_c| \nabla q|_c = |\Omega_c| \sum_{a \in c} q_a \nabla N_a|_c$   $\triangleright$  only the sign of the determinant survives
7:       for all corners  $a$  of  $\Omega_c$  do
8:          $\mathbf{g}_a \leftarrow \mathbf{g}_a + \gamma_c$   $\triangleright$  local to  $E$ 
9:       end for
10:    end for
11:    scatter: write nodes owned by  $E$  alone; stage nodes shared with other macro-elements
12:  end for
13:  reduce every shared node: sum its staged contributions in a fixed order
14:  for all nodes  $n$  do
15:     $\mathbf{g}_n \leftarrow \mathbf{W}_n^{-1} \mathbf{g}_n$ ,  $\mathbf{W}_n = \sum_{c \ni n} |\Omega_c|$   $\triangleright$  cached once per mesh
16:  end for
17:  return  $\mathbf{g}$ 
18: end function

```

Algorithm 2 Exact Jacobian action $y = J_{\text{exact}}(u, p)(v, q)$ (sscvfem.apply), Eq. (3)

Require: state (u, p) with its nodal gradient $\mathbf{g}^p$ (from the last update); direction (v, q)

```

1:  $\mathbf{g}^q \leftarrow \text{NODALGRADIENT}(q)$   $\triangleright$  the one extra pass; omitted, this is  $\mathbf{A}_{\text{frozen}}$ 
2:  $y \leftarrow 0$ 
3: for all macro-elements  $E$  in parallel do
4:   gather  $\mathbf{x}, u, p, \mathbf{g}^p, v, q, \mathbf{g}^q$  at the nodes of  $E$ 
5:   for all elements  $\Omega_c \subset E$  do
6:     viscous: traction of  $\nabla v|_c$  on each face of  $\Omega_c$ , into the momentum rows of its two nodes
7:     for all faces  $s$  of  $\Omega_c$ , separating nodes  $i$  and  $j$  do
8:        $\bar{\mathbf{u}} \leftarrow \frac{1}{2}(u_i + u_j)$ ,  $\bar{\mathbf{v}} \leftarrow \frac{1}{2}(v_i + v_j)$ ,  $c_s \leftarrow \text{Rhie-Chow coefficient at } |\bar{\mathbf{u}}|$ 
9:        $\text{corr} \leftarrow (p_j - p_i) - \frac{1}{2}(\mathbf{g}_i^p + \mathbf{g}_j^p) \cdot \mathbf{d}_s$   $\triangleright$  state, Eq. (1)
10:       $\dot{m}_s \leftarrow \rho \bar{\mathbf{u}} \cdot \mathbf{a}_s - c_s \text{corr}$ ; upwind split  $m^\pm$ ,  $\delta^\pm$  from its sign
11:       $\delta \dot{m}_s \leftarrow \rho \bar{\mathbf{v}} \cdot \mathbf{a}_s + c_s(q_i - q_j) + c'_s \text{corr} (\bar{\mathbf{u}} \cdot \bar{\mathbf{v}})$ 
12:       $+ \frac{1}{2} c_s \mathbf{d}_s \cdot (\mathbf{g}_i^q + \mathbf{g}_j^q)$   $\triangleright$  the  $K \mathbf{W}^{-1}\mathbf{G}$  term, Eq. (5)
13:       $\mathbf{f} \leftarrow \delta^+ \delta \dot{m}_s \mathbf{u}_i + m^+ v_i + \delta^- \delta \dot{m}_s \mathbf{u}_j + m^- v_j + \frac{1}{2}(q_i + q_j) \mathbf{a}_s$ 
14:       $y_i^u += \mathbf{f}$ ,  $y_j^u -= \mathbf{f}$ ,  $y_i^p += \delta \dot{m}_s$ ,  $y_j^p -= \delta \dot{m}_s$ 
15:    end for
16:    add the linearised flux of any domain-boundary faces of  $\Omega_c$ 
17:  end for
18:  scatter the local rows of  $E$  (owned nodes written, shared nodes staged)
19: end for
20: reduce every shared node in a fixed order
21:  $y^u += \rho |V| a_0 / \Delta t v$   $\triangleright$  transient term, only when  $\Delta t > 0$ 
22: return  $y$ 

```

Two properties of this structure matter for everything above. First, the only non-local step is the one marked $\mathbf{W}^{-1}\mathbf{G}$: remove line 1 of Algorithm 2 and every term reads the corners of a single element, which is $\mathbf{A}_{\text{frozen}}$ and what the element-wise assembly builds (SFEM_RC_EXACT_JAC=0 does exactly this). Second, the cost of exactness is one gradient pass per application, not a wider matrix: the red entries of Figure 4 are applied without ever being stored.

3.2 How the frozen Jacobian is applied and assembled

The frozen Jacobian A_{frozen} exists in two forms, and they are the same operator.

Applied, matrix-free. Algorithm 2 with line 1 removed and $\mathbf{g}^q = \mathbf{0}$. The state's gradient $\mathbf{g}^p$ is still read — it sets the mass flux and the upwind direction — but only as data: nothing differentiates through it. This is the action SFEM_RC_EXACT_JAC=0 selects.

Assembled, element by element. Because every remaining term reads the corners of one element, A_{frozen} is a sum of element matrices, and that is how the Vanka patches and every Galerkin coarse level are built (galerkin_assemble, galerkin_accumulate). Algorithm 3 writes the element matrix: its entries are the derivatives of the frozen face terms of Algorithm 2 with respect to the direction, with $\mathbf{w} = \delta^+ u_i + \delta^- u_j$ the upwind-weighted velocity of face s . Algorithm 4 assembles them, contracting each with the prolongation restricted to the element: at ratio $q = 1$ that restriction is the identity and the result is A_{frozen} itself, which the Vanka smoother reads; at $q > 1$ it is the Galerkin coarse operator $P^\top A_{\text{frozen}} P$. Assembly and action agree: at $q = 1$ the assembled matrix reproduces the matrix-free frozen action to $1.5\text{--}1.7 \times 10^{-16}$ on the closed box, the open outlet, the pressure port, the manufactured solution and the warped nozzle (SFEM_GMG_CHECK=1, job 4670644).

Algorithm 3 Frozen element matrix A_c of element Ω_c , 4×4 blocks $A_c[a, b]$ for corners a, b (cvfem_hex8_ns_upwind_jacobian_add_slots)

Require: state u, p and its nodal gradient $\mathbf{g}^p$ at the corners of Ω_c (data only)

```

1:  $A_c \leftarrow 0$ 
2: viscous: for each face of  $\Omega_c$ , the traction of  $\nabla v|_c$  gives the velocity–velocity blocks of its two
   nodes
3: for all faces  $s$  of  $\Omega_c$ , separating corners  $i$  and  $j$  do
4:    $\bar{\mathbf{u}}, c_s$ , corr,  $\dot{m}_s, m^\pm, \delta^\pm$  as in Algorithm 2 ▷  $\mathbf{g}^p$  read, not differentiated
5:    $\mathbf{b} \leftarrow \frac{1}{2}\rho \mathbf{a}_s + \frac{1}{2}c'_s \text{corr } \bar{\mathbf{u}}$  ▷  $\partial \delta \dot{m}_s / \partial v_i = \partial \delta \dot{m}_s / \partial v_j$ 
6:    $\mathbf{w} \leftarrow \delta^+ u_i + \delta^- u_j$ 
7:   for  $(k, \sigma) \in \{(i, +1), (j, -1)\}$  do ▷ row  $i$  gains the face flux, row  $j$  loses it
8:      $A_c[k, i]^{uu} += \sigma (\mathbf{w} \mathbf{b}^\top + m^+ I), \quad A_c[k, j]^{uu} += \sigma (\mathbf{w} \mathbf{b}^\top + m^- I)$ 
9:      $A_c[k, i]^{up} += \sigma (c_s \mathbf{w} + \frac{1}{2} \mathbf{a}_s), \quad A_c[k, j]^{up} += \sigma (-c_s \mathbf{w} + \frac{1}{2} \mathbf{a}_s)$ 
10:     $A_c[k, i]^{pu} += \sigma \mathbf{b}^\top, \quad A_c[k, j]^{pu} += \sigma \mathbf{b}^\top$ 
11:     $A_c[k, i]^{pp} += \sigma c_s, \quad A_c[k, j]^{pp} += -\sigma c_s$ 
12:   end for
13: end for
14: return  $A_c$  ▷ no  $K W^{-1} G$ : every column is a corner of  $\Omega_c$ 

```

Algorithm 4 Assembly into BSR: A_{frozen} at ratio $q = 1$, and $P^T A_{\text{frozen}} P$ at $q > 1$ (galerkin_assemble)

Require: ratio q ; fine constraint mask; the level's BSR pattern, derived from the lattice once and cached across Newton steps

- 1: values $\leftarrow 0$
- 2: **for all** chunks of macro-elements, in a fixed order **do**
- 3: **for all** macro-elements E in the chunk **in parallel do**
- 4: gather $\mathbf{x}, u, p, \mathbf{g}^p$ at the nodes of E ; hoist the element geometry
- 5: **for all** elements $\Omega_c \subset E$ **do**
- 6: $A_c \leftarrow \text{Algorithm 3}$
- 7: add the transient term $\rho a_0 / \Delta t \cdot |\Omega_c| / 8$ to each corner's velocity diagonal
- 8: add the Jacobian of any domain-boundary faces of Ω_c
- 9: zero the columns of constrained fine unknowns $\triangleright$ gives $P^T(AZ)P$
- 10: $Q \leftarrow$ prolongation restricted to Ω_c $\triangleright 8 \times 8$ weights; identity at $q = 1$
- 11: $C_E[a, b] += \sum_{i,j} Q_{ia} A_c[i, j] Q_{jb}$ $\triangleright$ coarse patch corners a, b ; 27 fixed slots
- 12: **end for**
- 13: **end for**
- 14: accumulate every slot of the chunk into its BSR block, through the cached inverse index
- 15: **end for**
- 16: **return** BSR(pattern, values) $\triangleright$ caller patches coarse identity rows

Set beside Algorithm 2, the difference is exactly one line and one kind of column. The exact action reads $\mathbf{g}^q$, a quantity computed from every element around a node; the frozen Jacobian never does, so each of its columns is a corner of the element being assembled, and the fixed 27-slot storage of Algorithm 4 is enough to hold it.

4 One coarse level

The coarse levels are Galerkin operators $P^T J P$, with P the bilinear interpolation from a lattice of half the resolution in each macro-element. Figure 5 shows one coarse row.

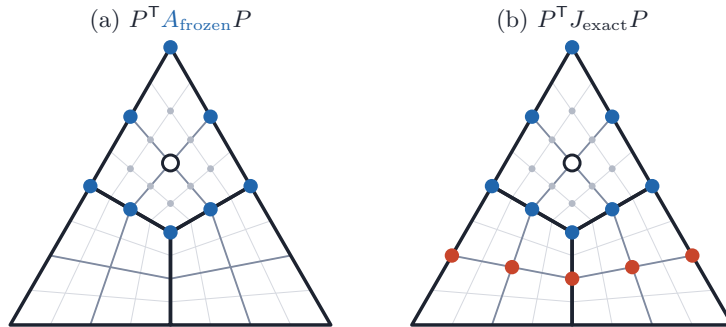


Figure 5: Row of the coarse node $I \circ$ at the centre of E_0 ; small grey dots $\bullet$ are the fine nodes where its basis is non-zero, thick grey lines the coarse lattice. (a) The coarse operator built today, $P^T A_{\text{frozen}} P$: 9 coarse nodes, 0 red. (b) The Galerkin operator of the exact Jacobian: 14 coarse nodes, 5 red. A coarse node on the shared edge reaches 9 and 16 coarse nodes respectively.

5 The same operators on a standard quad4 mesh

On a standard unstructured quad4 mesh every micro-cell is its own element, so an element matrix holds only the four nodes of one cell. Figure 6 draws the same rows on the same geometry with that element structure.

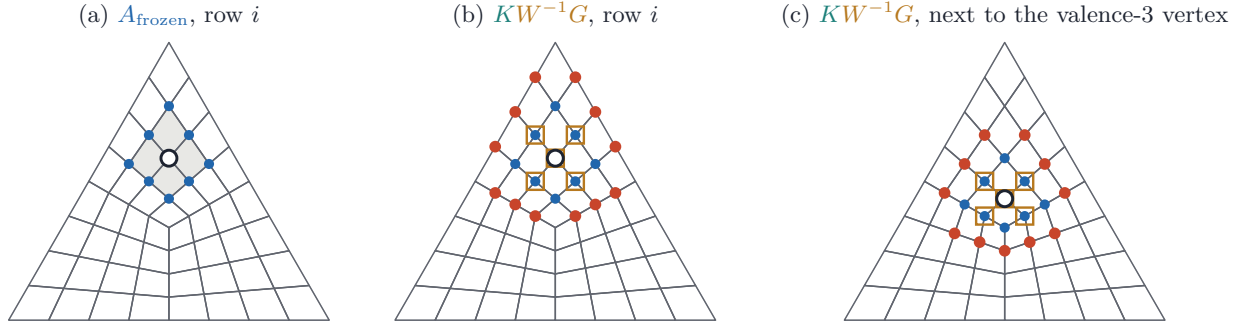


Figure 6: Every cell an element (all edges drawn as element boundaries). (a) A_{frozen} , row i $\bullet$: 9 nodes, 0 red — still exactly the nodes of the shaded cells, so an element-by-element assembly holds all of it. (b) $KW^{-1}G$, the same row i as Figure 4(a): 21 nodes, 12 red, against 0 red on the semi-structured mesh. (c) Next to the valence-3 vertex: 20 nodes, 11 red.

The operators are the same; only what counts as an element changed. With single-cell elements the whole second ring of $KW^{-1}G$ is red: no entry outside i 's own cells can be held by any element matrix, anywhere in the mesh. On the semi-structured mesh a macro-element matrix spans many cells, so the same entries are blue wherever the row's reach stays inside the macro-elements containing i . Red entries remain only in rows one node inside a macro-element boundary, where the gradient at a boundary node reaches one node into the neighbour.

6 Who else uses the two-ring operator

The two-ring *residual* is not specific to this solver. Any collocated discretisation that stabilises with a reconstructed gradient has it: the correction on a face reads gradients at its two ends, and each gradient reads the neighbours of its own node or cell. What varies between codes is what happens to that term when the equations are linearised. Three treatments appear in the literature: lag it, so the matrix stays compact; keep it exactly, but only in a matrix-free action; or linearise it exactly because the application demands an exact Jacobian.

Work	Discretisation	The two-ring term	In the linear system
Ansys CFX [2]	element-based FV (CVFEM), coupled u, v, w, p	Rhie–Chow/Majumdar “pressure-redistribution term”, a fourth derivative of pressure	not stated in the theory guide’s text; coupled AMG by additive correction
Nalu [3]	CVFEM, approximate projection	projected nodal gradient minus Gauss-point gradient, fourth order	lagged; “lagging the projected nodal gradient for pressure retains second order accuracy”
Sierra Fuego [4]	CVFEM, segregated projection	$(L_2 - D\tilde{\tau}_2 G)$ on the pressure	lagged: acts on p^k on the right-hand side of Eq. 3.794
Darwish, Sraj & Moukalled [5, 6]	cell-centred FV, coupled	interpolated cell gradient in the Rhie–Chow face velocity	moved to the right-hand side (their Eq. 27); compact coupled matrix, AMG
Nielsen & Anderson [7]	node-centred FV, compressible, discrete adjoint	reconstruction gradients in the second-order fluxes	exact linearisation needed; a nearest-neighbour one gave “many derivatives of incorrect sign”
Nejat & Ollivier-Gooch [8]	cell-centred FV, up to fourth order	high-order reconstruction	matrix-free Newton–GMRES, low-order Jacobian as preconditioner
Cardiff et al. [1]	cell-centred FV, solid mechanics	Rhie–Chow on displacement, least-squares cell gradients	exact action by finite differences (JFNK); compact Laplacian preconditioner
this solver	semi-structured CVFEM, coupled	nodal pressure gradient, $KW^{-1}G$	exact action analytically (<code>sscvfem.nodal.q-grad</code>); frozen Jacobian for GMG and Vanka

Two older results frame the matrix-free branch. Pernice and Tocci [9] recast SIMPLE as an operator on residuals and used it to precondition a Jacobian-free Newton–Krylov solver for incompressible Navier–Stokes; Geuzaine [10], as reported in [1], found a lower-order compact preconditioner gave the best performance with GMRES and ILU(k) for steady compressible flow. We did not find a collocated incompressible CVFEM or finite-volume solver that differentiates the averaged nodal pressure gradient analytically for Newton; the nearest are the finite-difference action of [1] and the exact linearisations built for discrete adjoints. That is the limit of what was found, not evidence that none exists.

6.1 Cardiff et al. (2025) in detail

This is the closest relative of the present solver [1], in cell-centred finite-volume solid mechanics (solids4foam, OpenFOAM, PETSc SNES).

The operator. The Rhie–Chow term of their Eq. 27 subtracts, on each face, a wide approximation of the normal gradient, $\Delta \cdot (\nabla u)_f$, from a compact two-cell one, $(u_N - u_P)/|d|$. The face gradient averages the two cells’ least-squares gradients, each built from that cell’s own neighbours, so a row reaches neighbours of neighbours. On a uniform one-dimensional mesh (their Appendix B) it reduces to

$$\frac{\alpha \bar{K}}{4\Delta x} (-u_{WW} + 4u_W - 6u_P + 4u_E - u_{EE}) = \frac{2}{3} \alpha \bar{K} \Delta x^2 \partial_x^3 U + O(\Delta x^3),$$

a five-point fourth difference that vanishes at second order — the one-dimensional counterpart of the 21-node (2D) and 81-node (3D) rows drawn above. They also show it is a jump between states extrapolated to the face midpoint from either side.

The solver. The Jacobian action is exact up to the finite-difference step, $Jv \approx [R(u + \epsilon v) - R(u)]/\epsilon$, so the two-ring term is in it. The preconditioner is the matrix of their segregated solver: inertia plus a compact Laplacian $\alpha \bar{K} (u_N - u_P)/|d|$ with $\alpha = 1$ — no material tangent, no coupling between components, no non-orthogonal correction — solved by MUMPS LU, ILU(5) or hypre BoomerAMG inside GMRES with restart 30. Linear solves stop at a relative 10^{-3} , Newton at 10^{-6} , globalised by a line search and a transient predictor.

What they measured.

- Faster than their segregated solver in every case that converged, typically by one to two orders of magnitude and at most $341\times$ (Cook’s membrane, 9 216 cells); fewer than five Newton iterations on average. Memory up to $3.6\times$ the segregated solver’s.
- Divergence on elastoplastic Cook’s membrane, on six of eight meshes at about 80 % of the load, attributed to the elastic compact preconditioner approximating the plastic tangent poorly.
- LU fastest in 2D and for moderate 3D sizes; BoomerAMG fastest for the largest 3D case, with near-ideal strong scaling to 128 cores.
- A weaker stabilisation, $\alpha = 0.01$, took the most GMRES iterations and failed on the ventricle case; $\alpha = 0.1$ and 1 performed alike.
- Their future work lists a comparison against a full-Jacobian method and reuse of the compact preconditioner for higher-order schemes.

Against this solver. Both apply the exact two-ring action and precondition with something narrower. The present solver differentiates the reconstruction analytically — one extra gradient pass rather than a residual evaluation, and no step size to choose — and its preconditioner drops only $KW^{-1}G$ while keeping the rest of the frozen Jacobian and the velocity–pressure coupling, measured 2–7 % from the exact action at $q = 1$. Their elastoplastic failure is the warning that transfers: an approximate preconditioner is sound until the true tangent moves away from it, so the size of the frozen–exact gap is worth tracking against Reynolds number and mesh rather than assumed small. Their α study also implies that the Rhie–Chow scale is a solver parameter as well as an accuracy one, which has not been measured here.

7 The lagged gradient (Nalu, Sierra Fuego)

Sandia’s CVPFEM codes meet exactly the operator of this note and resolve it the other way: they keep the matrix compact by *lagging* the gradient. This section explains the idea in the notation above, and what it does and does not cost.

7.1 The same stabilisation

Nalu’s continuity equation uses the stabilised mass flux [3]

$$\dot{m} = (\bar{\rho} \hat{u}_i + \tau G_i \bar{P} - \tau \partial_i \bar{P}) n_i dS,$$

where $G_i \bar{P}$ is a projected nodal pressure gradient — “an L_2 minimization between the discontinuous Gauss-point value and the continuous nodal value”; lumped, it is the Green–Gauss average $G_j \phi = (\sum \phi_{ip} A_j)/dV$ over the control volume. The difference between that nodal gradient interpolated to the integration point and the local gradient is Nalu’s “fourth order pressure stabilization term”.

Term for term this is Eq. (1): τ plays the role of c_s , the local gradient $\partial_i \bar{P}$ at the face is the compact part $c_s(p_j - p_i)$, and the projected gradient $G_i \bar{P}$ is $\mathbf{g}$, built by an averaging like $\mathbf{W}^{-1} \mathbf{G}$. Its Jacobian therefore has the same split, $J = \mathbf{A}_{\text{frozen}} + \mathbf{K} \mathbf{W}^{-1} \mathbf{G}$, and the same two-ring second term.

7.2 The idea: keep the compact part on the unknown, lag the wide part

The pressure system solves only the compact part for the new pressure and evaluates the wide part with the pressure of the previous iteration. Sierra Fuego writes the split out in its pressure equation [4] (Eq. 3.794),

$$-L_{\tau_1} \Delta p^{n+1/2} = -D(\hat{u}) - L_{\tau_1} p^k + (L_2 - D \tilde{\tau}_2 G) p^k + b,$$

where the Laplacian L_{τ_1} on the left is the compact operator on the unknown increment, and the term containing the projected gradient G acts on p^k , the previous iterate, on the right. In the terms of Figure 4: every entry of $\mathbf{K} \mathbf{W}^{-1} \mathbf{G}$, blue and red, leaves the matrix and becomes part of the right-hand side. The matrix keeps the compact element stencil — in Nalu’s words, CVFEM “results in the canonical 27-point stencil for a structured hexahedral mesh” — and so does everything built from it: the smoother, the AMG hierarchy, the storage.

Two things make this sound.

- *The solution does not change.* At convergence of the outer (Picard) iteration $p^{k+1} = p^k$, so the right-hand side term and the term it replaced are equal and the discrete equations satisfied are the fully stabilised ones. Lagging changes the iteration, not the answer.
- *The time accuracy survives.* Nalu’s verification on the convecting decaying Taylor vortex (100^2 , 200^2 and 400^2 meshes, Courant number 2, BDF2) finds that “lagging the projected nodal gradient for pressure retains second order accuracy”, whereas the pressure-free projection with second-order stabilisation is “confirmed to be first order accurate” [3].

Nalu also records the price in the place it shows: “due to the lagged coupling between the projected nodal gradient equation and the dofs, on meshes with high non-orthogonality, nonlinear residual convergence can be delayed” (said of its edge-based scheme).

7.3 What lagging costs, in this solver’s terms

The following derivation is ours. A Newton step that uses the frozen Jacobian but the full residual is exactly the lagged iteration: the residual $R(x^k)$ contains the gradient built from p^k , and the matrix $\mathbf{A}_{\text{frozen}}$ does not differentiate through it.

Algorithm 5 The lagged-gradient iteration, written as a frozen-Jacobian Newton step

Require: initial state $x^0 = (u^0, p^0)$

- 1: **for** $k = 0, 1, 2, \dots$ until $\|R(x^k)\|$ is small **do**
 - 2: $\mathbf{g}^{p,k} \leftarrow \text{NODALGRADIENT}(p^k)$ ▷ lagged: data for this step
 - 3: evaluate $R(x^k)$ with $\mathbf{g}^{p,k}$ in every face flux, Eq. (1)
 - 4: solve $\mathbf{A}_{\text{frozen}}(x^k) \delta x = -R(x^k)$ ▷ compact matrix; no $\delta \mathbf{g}$
 - 5: $x^{k+1} \leftarrow x^k + \delta x$
 - 6: **end for**
-

Near the solution x^* , with $R(x^k) \approx J(x^k - x^*)$ and $e^k = x^k - x^*$,

$$e^{k+1} = (I - \mathbf{A}_{\text{frozen}}^{-1} J) e^k = -\mathbf{A}_{\text{frozen}}^{-1} \mathbf{K} \mathbf{W}^{-1} \mathbf{G} e^k,$$

so the iteration converges linearly, at the spectral radius of $\mathbf{A}_{\text{frozen}}^{-1} \mathbf{K} \mathbf{W}^{-1} \mathbf{G}$, instead of quadratically. How much that costs depends on how large the lagged term is against the compact one.

In a time-stepping projection method with time-step-scaled τ , as in Nalu, nonlinear convergence already comes from outer Picard loops started from the previous time step, and the verification above shows the lag is harmless to the time accuracy there. In this solver’s steady continuation it was not: with the same residual, the frozen Jacobian needed 63 Newton steps and fell short of the target at $\text{Re} = 3200$, where the exact Jacobian took 5 (Poiseuille, 5 508 dof; commit 889d5a418).

7.4 Lagging versus freezing only the preconditioner

The distinction that decides the design is *where* the compact operator is used.

Treatment	Newton map	Matrix stored	Who pays
Lagged gradient (Nalu, Fuego; Alg. 5)	fixed-point, linear rate $\rho(A_{\text{frozen}}^{-1}KW^{-1}G)$	A_{frozen} , compact	outer iterations
Exact action, frozen preconditioner (this solver)	exact Newton, quadratic	A_{frozen} in Vanka and the coarse levels; J never stored	Krylov iterations; one gradient pass per action
Exact assembled Jacobian	exact Newton, quadratic	J , two-ring (21 nodes 2D, 81 in 3D)	memory and every sweep

Both of the first two store only A_{frozen} and keep every compact structure of Sections 3.2 and 4. The difference is that lagging puts the missing $KW^{-1}G$ into the *outer* iteration, where it sets the Newton convergence rate, while this solver applies it exactly in the Krylov operator (Algorithm 2), so the approximation lives only inside the preconditioner, where FGMRES absorbs it. That is why the frozen–exact gap measured by the multigrid gates costs this solver linear iterations rather than Newton steps.

8 Summary

Operator	Used by	Contains	Reach	
			2D	3D
matrix-free J_{exact}	FGMRES, Newton	both terms, every node	21	81
assembled A_{frozen}	Vanka, coarse levels	first term, every node	9	27
$P^T A_{\text{frozen}} P$	coarse levels today		9	27
$P^T J_{\text{exact}} P$	a consistent coarse space		21	81

The regular-lattice counts hold for a node whose neighbourhood lies inside one macro-element or inside a locally regular patch of them; near an irregular macro vertex the rows differ, as the figures show. Two consequences follow.

- A consistent coarse space needs $KW^{-1}G$ on the coarse levels, and its red entries couple nodes no macro-element contains, so it cannot come from the element-wise assembly. It has to be a sparse product over global node ids, $(P^T K)W^{-1}(GP)$.
- The fine-level smoother does not need it. Vanka is an approximation of the operator by construction, and reading the wider rows would cost the memory-bound sweep that dominates a run.

9 Smoothers that can use the exact operator

The solver already keeps the exact operator where Newton needs it: FGMRES applies J_{exact} , and the multigrid fine level is the same matrix-free operator, so every fine-level residual inside the cycle — including the one the stationary smoother forms between sweeps — is exact. The frozen A_{frozen} enters only through the Vanka patch solves and the Galerkin coarse levels. “Exact” for a smoother

can therefore mean two things: (a) it is driven by the exact residual, which every smoother below is; or (b) its local solves are built from the exact operator, which is what the third column records.

Smoother	Local solve built from	Exact (b)	Cost per smoothing step	Known here / risk
Vanka, frozen patches (current)	A_{frozen} restricted to each element, 8×8 Schur complement	no	row walk over 27-block rows, memory bound	multiplicative 0.63, additive 0.883 standalone at $\text{Re} = 1$; the default hook exists
Krylov-accelerated Vanka: GMRES(k) on J_{exact} , preconditioned by frozen Vanka	frozen patches as preconditioner; the Krylov polynomial is built on J_{exact}	through the polynomial	k exact applies + k Vanka sweeps; nonlinear cycle, absorbed by FGMRES	(SFEM_GMG_KSMOOTH) but uses BiCGStab, which should become GMRES/FGMRES; cheapest to try
Vanka, exact patches	$R_k J_{\text{exact}} R_k^T$: $K W^{-1} G$ entries among the corners as local path sums	yes	set-up reads one more ring of geometry; the multiplicative sweep needs 81-point rows, or an exact matrix-free residual per colour (≈ 8 applies per sweep)	more memory or more applies; local assembly of the two-ring entries is new code
Block smoothers (Braess-Sarazin, inexact Uzawa)	exact matrix-free blocks B , B^T , C (<code>blocks_exact_rc</code>), approximate F^{-1} and pressure Schur complement	yes (blocks)	a few block applies plus inner Schur solves	SIMPLE, the closest relative, measured 0.990; the velocity-column continuity block drops a term when the state pressure is non-zero and must be fixed first
Additive Schwarz on vertex stars (PCPATCH-like)	J_{exact} assembled on each star patch	yes	heavy set-up; one local factorisation per patch	robust in the literature; exact patch assembly needs neighbour data
Chebyshev / polynomial	matvecs of J_{exact} only	yes	degree $\times$ exact applies	needs eigenvalue bounds, unreliable for a non-symmetric indefinite saddle point
Point block-Jacobi	4×4 nodal blocks	possible	cheap	0.981 at $\text{Re} = 1$, divergent for $\omega \geq 0.5$: not a smoother here
ILU on assembled J_{exact}	the 81-point BSR matrix	yes	memory of the wide pattern; essentially sequential	reference only, not a many-core smoother

9.1 What the semi-structured discretisation adds

The rows above are written for a general sparse operator. On the semi-structured mesh two facts change which of them are affordable.

The current smoother is already macro-element-local. The multiplicative Vanka sweeps the elements of one macro-element in eight colours, on a private vector, and skips every column that lies outside that macro-element (`vanka_apply_mult`: “outside this element: additive”); corrections are then combined across macro-elements through the two-pass scatter. It never reads a coupling between two macro-elements.

The exact term's unstorable entries are exactly those couplings. Section 3 showed that the entries of $KW^{-1}G$ a macro-element matrix cannot hold (red in Figure 4) couple a row to a node outside every macro-element containing it. Every entry a macro-element-local sweep reads — row and column inside the same macro-element — is blue: storable, with W_n taken from the global mesh as data. So a macro-element-local smoother can be exact in the sense (b) *on everything it reads*, and what it gives up is what the current smoother already treats additively.

On the lattice those entries also have a fixed shape. Inside a macro-element the exact row of a node is the edge-neighbour star of K widened by the element box of G — 21 lattice offsets in 2D, 81 in 3D (the regular-lattice counts of the Summary table), truncated at the macro-element's faces — so it can be stored in fixed slots per lattice node, exactly as the Galerkin assembly stores 27 slots, with no indirection and no global pattern.

That storage is no longer hypothetical. The fine-level Vanka smoother now reads both its patch blocks and its multiplicative row walk from 27-slot macro-element stencils rather than from a global BSR, which was measured against the matrix form at identical iteration counts and is described in the companion note on the smoother. The first row of the table below therefore widens a path that exists, from 27 slots to 81, rather than introducing a new one.

Option on the lattice	What it uses	Exact (b)	Cost	Rests on
Macro-element exact Vanka	patch operators and the multiplicative row walk from the exact operator restricted to each macro-element, 81 fixed slots per lattice node (3D)	yes, for every entry read	$1.5\times$ the values the row walk streams today, not 81/27: the exact term differentiates through the pressure alone, so each of the 54 added offsets carries a column of four rather than a block of sixteen, giving $27 \times 16 + 54 \times 4 = 648$ against 432, in a sweep that is memory bound	the additive-between-macro-elements structure already in use
Macro-element Schwarz (block smoother, macro-elements as subdomains)	a local solve of the exact macro-element operator: inner Vanka sweeps or a direct solve on $(L+1)^3$ nodes	yes, inside each subdomain	one local solve per macro-element per smoothing step; parallel over macro-elements	the two-pass scatter for the additive combination
Line / plane smoothers along lattice axes	block solves along lattice lines (or planes) inside each macro-element; the exact operator along a line is pentadiagonal in 4×4 blocks	along the line	one block-banded solve per line; lines are free on the lattice	the classical remedy for convection dominance and anisotropy, cheap only because lines exist
Flow-aligned colour ordering	the eight lattice colours of each macro-element visited in the downstream direction of its mean velocity	as before	none	downstream ordering for convection-dominated flow; orthogonal to the other rows
Exact lattice stencils on coarse levels	Galerkin levels with the same fixed-slot storage, 81 slots instead of 27, built from $(P^T K) W^{-1} (GP)$	yes, locally	$3\times$ coarse storage; the cross-macro-element red entries at level 1 still need a global product	makes the coarse space consistent, which no smoother does

Of these, the macro-element exact Vanka is the one that keeps every structure the solver has — lattice slots, eight colours, the deterministic scatter — and changes only what the patches and the row walk are built from. It was measured before being built, and the measurement argues against it. Scaling the patch’s pressure couplings, which are exactly the entries it would correct, by 1 ± 0.05 moves the linear iteration count by one to two per cent, and by 1.10 under four per cent, with no consistent sign in either direction. The exact term is 2–7 % of the operator, so the most such a patch could recover is that couple of per cent, set against $1.5\times$ the bytes a memory-bound sweep streams and a fold map for the widened offsets that the 27-point pattern cannot key. The figures are in the companion note on the smoother.

Two remarks bound what any row can deliver. An exact smoother does not make the coarse space consistent: the Galerkin levels are still $P^T A_{\text{frozen}} P$, which differs from the exact operator at level 1 by 2–7 % on the cases measured. And `SFEM_RC_EXACT_JAC=0` does *not* isolate the smoother’s share of that, though it looks as though it should: it replaces the Krylov operator with one that

is not the Jacobian, so Newton itself degrades rather than the preconditioner. On the nozzle at 116212 dof it took 469 Newton steps over twelve continuation stages and still failed to converge, against 28 over six stages with the exact operator. Separating smoother mismatch from coarse-space mismatch requires the smoother to be varied while the operator is held exact, which is what the perturbation sweep above does.

References

- [1] P. Cardiff, D. Armfield, Ž. Tuković, I. Batistić. A Jacobian-free Newton–Krylov method for cell-centred finite volume solid mechanics. arXiv:2502.17217, 2025. <https://arxiv.org/abs/2502.17217>
- [2] Ansys, Inc. Ansys CFX-Solver Theory Guide, Release 2025 R1, Sections 11.1.1.7 and 11.2.
- [3] Sandia National Laboratories. Nalu 1.2.0 documentation: Theory — Pressure Stabilization, Discretization Approach, Coupling Approach; Verification — Convecting Decaying Taylor Vortex. <https://nalu.readthedocs.io>
- [4] Sandia National Laboratories. Sierra Fuego User Manual v5.26, Section 3.4.4, Discrete system of equations.
- [5] M. Darwish, I. Sraj, F. Moukalled. A coupled incompressible flow solver on structured grids. *Numerical Heat Transfer, Part B* 52(4) (2007) 353–371.
- [6] M. Darwish, I. Sraj, F. Moukalled. A coupled finite volume solver for the solution of incompressible flows on unstructured grids. *J. Comput. Phys.* 228 (2009) 180–201.
- [7] E. J. Nielsen, W. K. Anderson. Aerodynamic design optimization on unstructured meshes using the Navier–Stokes equations. AIAA Paper 98-4809, 1998.
- [8] A. Nejat, C. Ollivier-Gooch. A high-order accurate unstructured finite volume Newton–Krylov algorithm for inviscid compressible flows. *J. Comput. Phys.* 227 (2008) 2582–2609.
- [9] M. Pernice, M. D. Tocci. A multigrid-preconditioned Newton–Krylov method for the incompressible Navier–Stokes equations. *SIAM J. Sci. Comput.* 23 (2001) 398–418.
- [10] P. Geuzaine. Newton–Krylov strategy for compressible turbulent flows on unstructured meshes. *AIAA J.* 39 (2001) 528–531.